

PDN Master Formula & Plug-in Reference (Complete Uncompressed)

Module 0: Performance

Question Type: Calculating Speedup

Plug-in: S = Serial Time/Parallel Time.

Example Problem: Serial takes 10^6 s. Parallel takes 2×10^3 s.

Calculation: $1,000,000/2,000 = 500$.

Question Type: Calculating Efficiency

Plug-in: E = Speedup Result/Number of Processors.

Example Problem: With speedup 500 and 1024 processors.

Calculation: $500/1024 \approx 0.49$ (49%).

Question Type: Parallel Run-time with Overhead

Formula: $T_p = \frac{n^2}{p} + a \log_2(p)$.

Example: Overhead $a \log_2(p)$ causes efficiency to drop as p grows for fixed n .

Question Type: Amdahl's Law (Speedup Limit)

Plug-in: $S_{max} = 1/[(1 - f) + f/p]$.

Example Problem: If 90% is parallel ($f = 0.9$), what is max speedup?

Calculation: $1/(1 - 0.9) = 1/0.1 = 10\times$.

Question Type: Flynn's Taxonomy Details

SISD: 1 CPU (Sequential).

SIMD: 1 Instr/Multiple Data (GPU Vector). Same operation on different data.

MIMD: Multiple Instr/Multiple Data (Distributed Cluster). Different operations.

Module 1: OpenMP

Question Type: OpenMP Thread Management

Plug-in: rank (0 to N-1), total (count).

```
#include <omp.h>
int rank = omp_get_thread_num();
```

```
int total = omp_get_num_threads();
```

Question Type: Parallel Loop and Reduction

Code Snippet:

```
#pragma omp parallel for reduction(+:sum) private(x)
for(int i=0; i<n; i++) { sum += ... }
```

Example: Safely computing a Dot Product $\vec{A} \cdot \vec{B}$ with private index i .

Question Type: Trapezoidal Rule Integration

Step 1 (Plug-in): $h = (\text{End} - \text{Start})/n$.

Step 2 (Plug-in): Area = $\frac{h}{2} \times (\text{First} + \text{Last} + 2 \times \text{Sum of middle})$.

Example: Integrate $f(x)$ from 0 to 10 with 5 steps ($n = 5$).

Calculation: $h = 2$. Area

$= \frac{2}{2} \times [f(0) + f(10) + 2(f(2) + f(4) + f(6) + f(8))]$.

Question Type: Mutual Exclusion (Atomic vs Critical)

- **Atomic:** Fast hardware update for specific ops ($++$, $+=$).
- **Critical:** High-level lock for complex code blocks.

Question Type: Mutual Exclusion Scalability

Comparison: Locks (Parallel per bin) vs. Critical (Serialized).

Example: 100-bin histogram update. ****Array of Locks**** is faster.

Question Type: Identifying False Sharing

Concept: Threads updating data on same ****64-byte Cache Line****.

Solution: Use ****Private variables**** or ****Padding**** to break lines.

Module 2: CUDA & GPUs

Question Type: Execution Configuration

Concept: Specifies grid/block dimensions for launch ($\lll G, B \ggg$).

Question Type: Grid and Block Dimensions

Plug-in: $G = \lceil \text{Total Data} / \text{Block Size} \rceil$.

Example Problem: $N = 2000$ additions, Block Size 256.

Calculation: $\lceil 2000 / 256 \rceil = 8$ blocks.

Question Type: Global Coordinate Calculation

Plug-in: $GlobalX = blockIdx.x \times blockDim.x + threadIdx.x$.

Example: $blockIdx = 4, threadIdx = 11, blockDim = 32$.

Calculation: $4 \times 32 + 11 = 128 + 11 = 139$.

Question Type: Local Thread Index from Global

Plug-in: $threadIdx.x = GlobalCol \pmod{blockDim.x}$.

Example: Pixel at $[col = 91]$, Block Size 16.

Calculation: $91 \pmod{16} = 11$.

Question Type: Block Index from Global Coord

Plug-in: $blockIdx.x = \lfloor GlobalCol / blockDim.x \rfloor$.

Calculation: $\lfloor 91/16 \rfloor = 5$.

Question Type: 2D and 3D Indexing

2D Row: $r = blockIdx.y \times blockDim.y + threadIdx.y$.

3D Plane: $z = blockIdx.z \times blockDim.z + threadIdx.z$.

Question Type: Stencil Blur Computation (5-Point)

Plug-in: $P_{out} = \frac{Target + Above + Below + Left + Right}{5}$.

Rule: Special handling needed for out-of-domain boundaries.

Question Type: Total Threads in Multi-D Grid

Plug-in: $Total = (G_x B_x) \times (G_y B_y) \times (G_z B_z)$.

Example: 2D Grid $G = (250, 188)$, $B = (8, 8)$.

Calculation: $(250 \times 8) \times (188 \times 8) = 3,008,000$.

Question Type: Calculating Idle Threads

Plug-in: $Idle = Total\ Launched - Actual\ Data\ Size$.

Calculation: $2048 - 2000 = 48$ idle threads.

Question Type: Tree Reduction Complexity and Stride

Iterations: $\log_2(N)$.

Stride (Interleaved): 2^{k-1} . **Stride (Sequential):** $N/2^k$.

Divergence Rule: Interleaved causes divergence in all 32 warps after step 5.

Key Point: GPU Memory Hierarchy

Registers (Fastest) $\rightarrow$ Shared (Block-level) $\rightarrow$ Global (Slow).

Question Type: Shared Memory Capacity Needed

Plug-in: $Mem = Tile^2 \times Size \times 2 \times BlocksPerSM$.

Example: 16×16 float, 8 blocks per SM.

Calculation: $16 \times 16 \times 4 \times 2 \times 8 = 16,384$ bytes (16KB).

Question Type: Compute-to-Memory Ratio (Tiling)

Without Tiling: Ratio is $**1**$ (Memory bound).

With Tiling: Ratio is $**TILE_WIDTH**$ (Arithmetic bound).

Example: 250 GFLOPS $\rightarrow$ 4 TFLOPS (Width 16).

Module 3: MPI

Question Type: MPI Performance Scaling

Serial Sum: $Time = (P - 1) \times Addition\ Delay$.

Tree Sum: $Time = \log_2(P) \times Message\ Delay$.

Question Type: MPI Neighbor ID Calculation

Right Neighbor: $(rank + 1) \pmod{size}$.

Left Neighbor: $(rank + size - 1) \pmod{size}$.

Rule: Use `MPI_Sendrecv` to avoid neighbor deadlocks.

Question Type: Message Transmission Time

Plug-in: $Time = Latency + (Message\ Size / Bandwidth)$.

Question Type: Derived Data Type Benefit

Plug-in: $Latency\ Savings = (Sends\ Consolidated - 1) \times Latency$.

Example: Combining 3 sends into 1 saves $**2 \times Latency**$.

Module 4: MapReduce

Question Type: Mapper and Reducer Data Flow

Mapper: Emits (key, value). **Reducer:** Aggregates list per key.

Question Type: Combiner Capability (Avg vs Median)

Average Plug-in: Combiner must yield (sum, count).

Median Plug-in: NO combiner possible. High shuffle cost.

Module 5: Network

Question Type: Transmission vs Propagation Delay

Trans Plug-in: $d_{trans} = L/R$. **Prop Plug-in:** $d_{prop} = d/s$.

Calculation: 40,000 bits/4Mbps = 10ms.

Question Type: Total End-to-End Delay

Template: Delay = Sum of all hop Trans/Prop + Queuing.

Calculation: $(d_t + d_p)_{A \rightarrow R} + d_{queue_R} + (d_t + d_p)_{R \rightarrow B}$.

Question Type: Bottleneck Throughput

Plug-in: Throughput = $\min(R_1, R_2, \dots, R_n)$.

Calculation: $\min(10, 2, 100) = 2\text{Mbps}$.

Question Type: Internet Layer Mapping

Plug-in Stack: App (Msg) $\rightarrow$ Trans (Seg) $\rightarrow$ Net (Datagram) $\rightarrow$ Link (Frame) $\rightarrow$ Phys (Bits).

Device Placement: Routers $\rightarrow$ Network Layer. Switches $\rightarrow$ Link Layer.

Question Type: TCP vs UDP Service Classes

- **Reliable Transfer:** TCP only.
- **Throughput/Timing:** Neither.
- **Confidentiality:** Neither (requires SSL/TLS).

Question Type: Destination Sockets

IP Address: Identifies host. **Port:** Identifies process.

Socket: The "door" between app and transport layers.

Question Type: Packet Movement Logic

Vertical: Down the stack at source, up at destination.

Horizontal: Via Link and Physical layers between nodes.